

El TAD Cola por dentro

Por un extremo se entra y por el otro se sale

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Explicar cómo la cola sobre arreglo da la vuelta con el residuo y por qué eso la hace $\Theta(1)$ en las dos operaciones (OA1, OA2).
- Mostrar que una cola necesita los dos extremos baratos, y que ninguna lista de un solo extremo barato sirve (OA1, OA3).
- Construir la cola sobre la lista enlazada con puntero al último, y directamente sobre nodos, con encolar, desencolar y frente en $\Theta(1)$ (OA2, OA3).
- Dar el costo en tiempo y en espacio de cada implementación y decidir cuál conviene según se conozca o no el número máximo de elementos (OA1, OA5).

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación
- 6 Ejercicios

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación
- 6 Ejercicios

El arreglo que da la vuelta I

La cola atiende por los dos extremos: por uno entra y por el otro sale. Lo primero que entra es lo primero que sale.

```
class Cola {  
private:  
    Elemento datos[CAPACIDAD];  
    int inicio;    // posicion del frente  
    int n;  
public:  
    void encolar(Elemento e) {  
        assert(n < CAPACIDAD);  
        datos[(inicio + n) % CAPACIDAD] = e;  
        n = n + 1;  
    }  
}
```

El arreglo que da la vuelta II

```
// exige !vacía()
void desencolar() {
    assert(n > 0);
    inicio = (inicio + 1) % CAPACIDAD;
    n = n - 1;
}

// exige !vacía()
Elemento frente() {
    assert(n > 0);
    return datos[inicio];
}
```

Desencolar no mueve ni un elemento: adelanta `inicio`. Las casillas que quedan atrás se reutilizan porque el residuo las trae de vuelta.

El arreglo que da la vuelta III

Con CAPACIDAD = 6, tras encolar seis y desencolar tres

inicio = 3, $n = 3$

			d	e	f
0	1	2	3	4	5

Encolar ahora escribe en $(3 + 3) \bmod 6 = 0$: la cola sigue en la casilla 0 aunque empiece en la 3.

Lo que arrastra

Igual que la pila: CAPACIDAD se fija al compilar, el elemento 1001 no entra y las casillas vacías quedan reservadas.

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación
- 6 Ejercicios

¿Sirve la receta de la pila? I

La pila se construyó sobre la lista usando un solo extremo, y con la enlazada quedó en $\Theta(1)$ porque insertar y eliminar al frente son dos punteros.

Pregunta

Si la cola entra por un extremo y sale por el otro, ¿qué lista sirve?
¿Alcanza con la enlazada simple?

Las cuatro combinaciones

Sobre la lista hay dos decisiones independientes: por cuál extremo entra y por cuál sale. Y hay dos listas de abajo. Lo que importa es que *los dos* extremos sean baratos a la vez, no uno.

Solo una sirve I

Encolando y desencolando n veces, contando los elementos que se mueven en el arreglo y los nodos que se visitan en la enlazada:

n	arreglo, entra al final	arreglo, entra al frente	enlazada simple	enlazada con ultimo
100	4 950	4 950	4 851	0
1000	499 500	499 500	498 501	0
10000	49 995 000	49 995 000	49 985 001	0

Por qué las tres primeras son cuadráticas

En el arreglo, entrar por el final deja salir por el frente corriendo todo, y entrar por el frente obliga a correr todo al entrar: se escoja lo que se escoja, una de las dos paga $\Theta(n)$. En la enlazada simple salir por el frente es gratis, pero entrar por el final exige caminar hasta allá.

Solo una sirve II

Y por qué la última es cero

La lista con puntero al último pone en $\Theta(1)$ por el final y quita en $\Theta(1)$ por el frente. Es la única de las cuatro donde los dos extremos de la cola caen sobre las dos operaciones baratas.

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada**
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación
- 6 Ejercicios

Cinco llamadas I

El frente es la posición 0 y el final es el último. La lista con puntero al último ya tiene las dos operaciones en $\Theta(1)$.

```
class Cola {  
private:  
    Lista l;  
  
public:  
    void encolar(Elemento e) {  
        l.agregar(e);  
    }  
    // exige !vacía()  
    void desencolar() {  
        assert(!vacía());  
        l.eliminar(0);  
    }  
}
```

Cinco llamadas II

```
// exige !vacía()  
Elemento frente() {  
    assert(!vacía());  
    return l.obtener(0);  
}  
int tamaño() {  
    return l.tamaño();  
}  
bool vacía() {  
    return l.vacía();  
}  
};
```

Cinco llamadas III

Salida

Encolando 5, 8 y 2, desencolando una vez y encolando 9: 5 3 / 8
2 / 8 2 9 / 1 1 0

La última línea es la cola que se vació y volvió a servir. Ese es el caso donde la implementación se rompe si no se cuida.

Por qué al revés no funciona I

La otra decisión parece simétrica: que entre por el frente y salga por el final.

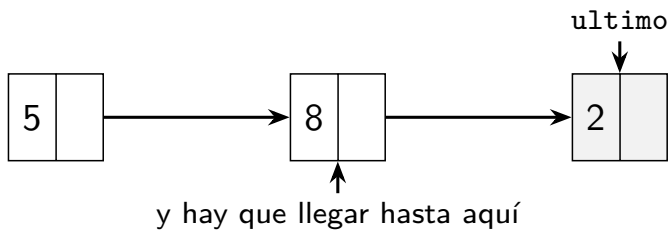
Pregunta

Con la misma lista con puntero al último, ¿cuánto costaría esa versión?

Encolar sí, desencolar no

Insertar en la posición 0 son dos punteros: $\Theta(1)$. Pero eliminar el último exige reescribir el campo siguiente *del penúltimo*, y guardar el último no dice quién es el penúltimo. Hay que recorrer: $\Theta(n)$.

Por qué al revés no funciona II



La regla que queda

Guardar un puntero a un extremo sirve para **poner** ahí, no para **quitar** de ahí. Quitar necesita llegar al anterior, y eso solo lo da el campo anterior de la doblemente enlazada.

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio**
- 5 Lo que cobra cada operación
- 6 Ejercicios

La cola directa sobre nodos I

La cola solo toca los dos extremos, así que se puede escribir la cadena guardando los dos.

```
class Cola {  
private:  
    Nodo *cabeza;  
    Nodo *ultimo;  
    int n;  
public:  
    Cola() {  
        cabeza = NULL;  
        ultimo = NULL;  
        n = 0;  
    }  
}
```

La cola directa sobre nodos II

```
void encolar(Elemento e) {  
    Nodo *nuevo = new Nodo;  
    nuevo->dato = e;  
    nuevo->siguiente = NULL;  
    if (ultimo == NULL) {  
        cabeza = nuevo;  
    } else {  
        ultimo->siguiente = nuevo;  
    }  
    ultimo = nuevo;  
    n = n + 1;  
}
```

La cola directa sobre nodos III

Con la cola vacía no hay último al que colgarse, así que el nodo nuevo es a la vez cabeza y último. La pregunta se hace sobre ultimo, que es el extremo por el que se entra, y eso obliga a que ultimo diga la verdad siempre:

`cabeza = NULL  $\iff$  ultimo = NULL.`

La cola directa sobre nodos IV

```
// exige !vacía()
void desencolar() {
    assert(!vacía());
    Nodo *muerto = cabeza;
    cabeza = cabeza->siguiente;
    if (cabeza == NULL) {
        ultimo = NULL;
    }
    delete muerto;
    n = n - 1;
}
```

La cola directa sobre nodos V

El caso que rompe la cola

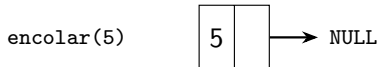
Al sacar el único elemento, cabeza queda en NULL pero ultimo seguiría apuntando al nodo que acaba de liberarse. El siguiente encolar escribiría ultimo->siguiente sobre memoria devuelta. Por eso ultimo vuelve a NULL en el mismo paso.

La traza I

`encolar(5), encolar(8), desencolar()` y `desencolar()`.

La traza II

cabeza y ultimo



cabeza

ultimo



cabeza y ultimo



desencolar()

la cola queda vacía: cabeza y ultimo en NULL

La traza III

Sale por donde está cabeza y entra por donde está ultimo. Los dos extremos se tocan sin recorrer nada.

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación**
- 6 Ejercicios

Las cinco formas de tener una cola I

Operación	Arreglo circular	Lista sobre arreglo	Lista enlazada simple	Lista con ultimo	Directa sobre nodos
encolar(e)	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
desencolar()	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
frente()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
tamano(), vacia()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
espacio de la estructura	$\Theta(\text{CAPACIDAD})$	$\Theta(\text{CAPACIDAD})$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
espacio por elemento	el dato	el dato	dato y puntero	dato y puntero	dato y puntero
techo de elementos	CAPACIDAD	CAPACIDAD	la memoria	la memoria	la memoria

Las cinco formas de tener una cola II

Por qué

- El arreglo circular no mueve nada: adelanta inicio y calcula el residuo. Es $\Theta(1)$ en las dos, y paga con un techo fijo.
- La lista sobre arreglo entra barato por el final y sale caro por el frente, porque sacar el primero corre todo lo demás.
- La enlazada simple es lo contrario: sale barato por el frente y entra caro por el final, porque hay que caminar.
- Guardar el último arregla justo esa mitad y deja las dos en $\Theta(1)$, sin techo.

Si se sabe de antemano cuántos elementos caben, el arreglo circular gana: no paga un puntero por elemento. Si no se sabe, la enlazada con el último es la única que da las dos operaciones baratas sin límite.

Para el juez: UVa 540 — Team Queue I

UVa 540 — Team Queue,

<https://onlinejudge.org/external/5/540.pdf>

Envío: [https://onlinejudge.org/index.php?option=com_onlinejudge
&Itemid=25&page=submit_problem&problemid=481](https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=481)

Qué pide

Una fila donde cada persona pertenece a un equipo. Quien llega se pone detrás del último de su equipo si ya hay alguien de su equipo en la fila, y si no, al final de todo. Hay que procesar ENQUEUE, DEQUEUE y STOP e imprimir quién sale cada vez.

Entrada, salida y límites

Hasta 1000 equipos por caso y hasta 200 000 órdenes. El archivo termina con un escenario de cero equipos. Cada escenario lleva encabezado `Scenario #k` y una línea en blanco después.

Para el juez: UVa 540 — Team Queue II

Cómo atacarlo con lo de hoy

Una cola de equipos y una cola por equipo. Al encolar a alguien se mira si su equipo ya tiene cola: si la tiene, entra ahí; si no, el equipo entra a la cola de equipos y la persona a la suya. Al desencolar sale el frente de la cola del equipo que está al frente, y si ese equipo queda vacío, sale de la cola de equipos.

Para el juez: UVa 540 — Team Queue III

Las trampas

- Con 200 000 órdenes, una cola que cueste $\Theta(n)$ por operación da $\Theta(n^2)$ y no entra en el tiempo. Las dos operaciones tienen que ser $\Theta(1)$.
- Hay que saber en qué equipo está cada persona sin recorrer: una tabla de la persona a su equipo, que es justamente lo que viene en el tema siguiente.
- Un equipo que se vacía y vuelve a recibir gente entra otra vez al final de la cola de equipos, no a su puesto anterior.

Contenido I

- 1 La cola que ya tienen
- 2 La cola necesita los dos extremos
- 3 La cola sobre la lista enlazada
- 4 Sin la lista de por medio
- 5 Lo que cobra cada operación
- 6 Ejercicios

Ejercicio en parejas: la cola circular de nodos I

Cierre la cadena de la cola sobre sí misma, como la lista circular, y guarde **solo** un puntero al último.

Las preguntas

- 1 Con ultimo guardado y la cadena circular, ¿cómo se llega al frente?
- 2 Escriba encolar y desencolar. ¿Cuánto cuestan?
- 3 ¿Cuántos punteros guarda la clase, contra los dos de la versión de hoy?
- 4 ¿Qué caso especial queda?

Ejercicio en parejas: la cola circular de nodos II

Lo que debe salir

El frente es `ultimo->siguiente`: un paso. Encolar empalma el nodo nuevo entre `ultimo` y el frente, y mueve `ultimo`; desencolar empalma `ultimo->siguiente` sobre el que sigue. Las dos son $\Theta(1)$ y la clase guarda *un* puntero en vez de dos.

El caso especial es el de siempre: con un solo nodo, ese nodo se apunta a sí mismo, y al sacarlo `ultimo` vuelve a `NULL`.

Ejercicio en parejas: la cola con dos pilas I

Implemente una cola usando únicamente dos pilas y sus operaciones. `encolar` apila en la primera; `desencolar` saca de la segunda, y cuando la segunda está vacía, vuelca toda la primera en ella antes de sacar.

Las preguntas

- 1 ¿Por qué volcar una pila en otra deja los elementos en el orden de la cola?
- 2 ¿Cuánto cuesta un `desencolar` que encuentra la segunda vacía? ¿Y uno que la encuentra con elementos?
- 3 Encolando n y desencolando n , ¿cuántas operaciones de pila se hacen en total?
- 4 ¿Cuánto cuesta entonces cada operación en promedio?

Ejercicio en parejas: la cola con dos pilas II

Lo que debe salir

Volcar invierte, y la pila ya tenía el orden invertido respecto de la llegada: dos inversiones dejan el orden original. Un desencolar que vuelca cuesta $\Theta(k)$ con k elementos en la primera; uno que no vuelca, $\Theta(1)$.

Cada elemento se apila y se desapila exactamente dos veces en toda su vida, así que n encolar y n desencolar hacen $4n$ operaciones de pila. Repartidas entre las $2n$ llamadas, salen dos por llamada: el costo *amortizado* es $\Theta(1)$, aunque una llamada suelta pueda costar $\Theta(n)$.

Ejercicio en parejas: la cola con dos pilas III

La diferencia que importa

El peor caso de una operación y el costo promedio de una serie de operaciones son dos cosas distintas. Aquí el peor caso es $\Theta(n)$ y la serie completa es lineal.

Propuestos para la casa I

- 1 **El residuo al revés.** En la cola de arreglo, ¿qué pasa si encolar escribe en `datos[n]` en vez de `datos[(inicio + n) % CAPACIDAD]`? Dé una secuencia que falle.
- 2 **La cola doble.** Una cola que deje entrar y salir por los dos extremos. ¿Qué lista hace falta para que las cuatro operaciones sean $\Theta(1)$?
- 3 **Sin contador.** Si la cola de nodos no guardara `n`, ¿cómo se sabe si está vacía? ¿Y cuántos elementos tiene?
- 4 **La cola de prioridad, a mano.** Una cola donde sale siempre el menor. Implementada sobre la lista enlazada, ¿cuánto cuesta encolar y cuánto desencolar? ¿Y si se guarda ordenada?
- 5 **Invertir una cola.** Déle la vuelta a una cola usando solo una pila auxiliar y las operaciones de los dos contratos. ¿Cuánto cuesta?

Resumen I

- La cola de arreglo da la vuelta con el residuo: desencolar adelanta inicio sin mover elementos, así que las dos operaciones son $\Theta(1)$ a cambio de un techo fijo.
- Una cola exige los dos extremos baratos a la vez. De las cuatro combinaciones sobre lista, solo la enlazada con puntero al último lo logra: las otras tres son cuadráticas al encolar y desencolar n veces.
- Guardar un puntero a un extremo sirve para poner ahí, no para quitar de ahí. Quitar necesita llegar al anterior, y eso solo lo da el campo anterior.
- Al sacar el único elemento hay que devolver ultimo a NULL, o el siguiente encolar escribe sobre memoria liberada.

Resumen II

Lo que sigue

Los ejercicios sobre las tres implementaciones, y el tercer algoritmo de ordenamiento: quicksort, que parte alrededor de un elemento en vez de por la mitad.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 8, colas: circular, con listas enlazadas y sus variantes.
-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022. Sección 10.1, pilas y colas; sección 17.1, análisis amortizado.
-  N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.